

Cheat Sheet - React Props

Lezione: **823 - React: Props**

Contesto: dopo JSX, componenti e iterazioni con `.map()`; prima di `useState`.

Obiettivo della Lezione

Le props servono a rendere i componenti React **riutilizzabili**.

Invece di creare tanti componenti quasi uguali, creo un componente generico e gli passo dall'esterno i dati che cambiano.

```
function ComicCard(props) {  
  return (  
    <article className="comic-card">  
      <img src={props.image} alt={props.title} />  
      <h3>{props.title}</h3>  
    </article>  
  );  
}
```

Uso del componente:

```
<ComicCard  
  image="/img/superman.jpg"  
  title="Superman"  
>
```

Idea chiave:

```
componente = struttura riutilizzabile  
props = dati che personalizzano quella struttura
```

Cosa Sono le Props

Le **props** sono informazioni passate da un componente padre a un componente figlio.

```
function App() {  
  return <Welcome name="Emanuele" />;  
}  
  
function Welcome(props) {
```

```
    return <h1>Ciao {props.name}</h1>;  
  }
```

Risultato:

Ciao Emanuele

Da ricordare:

- le props arrivano come oggetto;
- sono passate come attributi JSX;
- sono lette nel componente figlio;
- il componente figlio non dovrebbe modificarle;
- il flusso dei dati va dal padre al figlio.

Props con Destructuring

In React spesso si usa il destructuring per scrivere componenti piu leggibili.

Versione base:

```
function ProductCard(props) {  
  return (  
    <article>  
      <h3>{props.name}</h3>  
      <p>{props.price} euro</p>  
    </article>  
  );  
}
```

Versione con destructuring:

```
function ProductCard({ name, price }) {  
  return (  
    <article>  
      <h3>{name}</h3>  
      <p>{price} euro</p>  
    </article>  
  );  
}
```

Il risultato e lo stesso. Cambia solo la leggibilita.

Props e Tipi di Dati

In JSX posso passare stringhe, numeri, booleani, array, oggetti e funzioni.

Stringa:

```
<Button label="Compra ora" />
```

Numero:

```
<ProductCard price={19.99} />
```

Booleano:

```
<Alert isVisible={true} />
```

Array:

```
<TagList tags={["React", "JSX", "Props"]} />
```

Oggetto:

```
<UserCard user={{ name: "Ada", role: "Developer" }} />
```

Regola pratica:

```
stringhe semplici -> virgolette  
valori JavaScript -> graffe
```

```
<Card title="Titolo statico" count={3} />
```

Props e Map

Le props diventano molto utili insieme a `.map()`, per trasformare un array di dati in una lista di componenti.

```
const comics = [  
  {  
    id: 1,
```

```
    title: "Action Comics",
    image: "/img/action-comics.jpg",
  },
  {
    id: 2,
    title: "Batman",
    image: "/img/batman.jpg",
  },
];
```

```
function ComicsList() {
  return (
    <section className="comics-list">
      {comics.map((comic) => (
        <ComicCard
          key={comic.id}
          title={comic.title}
          image={comic.image}
        />
      ))}
    </section>
  );
}
```

```
function ComicCard({ title, image }) {
  return (
    <article className="comic-card">
      <img src={image} alt={title} />
      <h3>{title}</h3>
    </article>
  );
}
```

Da notare:

- `key` serve a React per identificare gli elementi della lista;
- `key` non viene ricevuta come prop dal componente;
- `title` e `image` invece arrivano davvero a `ComicCard`.

Children

`children` è una prop speciale: contiene quello che scrivo tra tag di apertura e tag di chiusura del componente.

```
function Card({ children }) {
  return (
    <div className="card">
```

```
    {children}
  </div>
);
}
```

Uso:

```
<Card>
  <h2>Titolo</h2>
  <p>Testo della card.</p>
</Card>
```

Risultato concettuale:

```
<div className="card">
  <h2>Titolo</h2>
  <p>Testo della card.</p>
</div>
```

`children` è utile quando il componente deve fornire una struttura comune, ma il contenuto interno può cambiare.

Esempio:

```
function Section({ title, children }) {
  return (
    <section>
      <h2>{title}</h2>
      <div>{children}</div>
    </section>
  );
}
```

```
<Section title="Fumetti in evidenza">
  <ComicCard title="Batman" image="/img/batman.jpg" />
  <ComicCard title="Superman" image="/img/superman.jpg" />
</Section>
```

Conditional Rendering

Il conditional rendering permette di mostrare contenuti diversi in base alle props.

Con operatore ternario:

```
function Badge({ isNew }) {  
  return (  
    <span className="badge">  
      {isNew ? "Nuovo" : "Gia visto"}  
    </span>  
  );  
}
```

Con `&&`, utile quando voglio mostrare qualcosa solo se una condizione e vera:

```
function ComicCard({ title, image, isNew }) {  
  return (  
    <article className="comic-card">  
      <img src={image} alt={title} />  
      <h3>{title}</h3>  
      {isNew && <span className="badge">New</span>}  
    </article>  
  );  
}
```

Con return anticipato:

```
function Alert({ message }) {  
  if (!message) {  
    return null;  
  }  
  
  return <p className="alert">{message}</p>;  
}
```

Da ricordare:

- `return null` significa "non renderizzare nulla";
- il ternario va bene quando ho due alternative;
- `&&` va bene quando devo mostrare qualcosa solo in un caso;
- evitare condizioni troppo lunghe dentro il JSX.

Classi CSS Dinamiche

Le props possono controllare anche le classi CSS.

```
function Button({ label, variant }) {  
  return (  
    <button className={`button button-${variant}`}>  
      {label}  
    </button>  
  );  
}
```

```
    </button>
  );
}
```

Uso:

```
<Button label="Salva" variant="primary" />
<Button label="Annulla" variant="secondary" />
```

HTML generato idealmente:

```
<button class="button button-primary">Salva</button>
<button class="button button-secondary">Annulla</button>
```

Per valori booleani:

```
function Card({ title, isHighlighted }) {
  const className = isHighlighted ? "card highlighted" : "card";

  return (
    <article className={className}>
      <h3>{title}</h3>
    </article>
  );
}
```

Componenti Core, Riutilizzabili e Dumb

Nella lezione potrebbe comparire una distinzione importante.

Componenti Core

Sono i componenti principali dell'applicazione o della pagina.

Esempi:

- App
- Header
- Main
- Footer
- ComicsList

Di solito:

- organizzano il layout;

- importano dati o altri componenti;
- decidono quali sezioni mostrare;
- possono contenere logica di composizione.

Esempio:

```
function App() {  
  return (  
    <>  
      <Header />  
      <main>  
        <ComicsList />  
      </main>  
      <Footer />  
    </>  
  );  
}
```

Componenti Riutilizzabili di Progetto

Sono componenti pensati per essere usati in più parti dello stesso progetto.

Esempi:

- Button
- Card
- Section
- Navbar
- Logo

Di solito:

- ricevono dati tramite props;
- hanno una struttura generica;
- evitano duplicazione di JSX e CSS;
- restano coerenti con lo stile del progetto.

Esempio:

```
function Button({ label, variant }) {  
  return (  
    <button className={`button button-${variant}`}>  
      {label}  
    </button>  
  );  
}
```

Dumb Component

Un dumb component è un componente "stupido" nel senso buono: non decide da solo cosa mostrare, ma viene governato dall'esterno tramite props.

Esempio:

```
function ComicCard({ title, image, price, isAvailable }) {  
  return (  
    <article className="comic-card">  
      <img src={image} alt={title} />  
      <h3>{title}</h3>  
      <p>{price}</p>  
      {isAvailable && <span>Disponibile</span>}  
    </article>  
  );  
}
```

ComicCard non conosce l'array completo dei fumetti e non decide quali fumetti esistono. Riceve solo i dati necessari per renderizzare una singola card.

Questo lo rende:

- facile da riutilizzare;
- facile da testare mentalmente;
- facile da spostare;
- facile da leggere.

Esempio Completo Course-Level

```
const comics = [  
  {  
    id: 1,  
    title: "Action Comics",  
    image: "/img/action-comics.jpg",  
    price: "$19.99",  
    isAvailable: true,  
  },  
  {  
    id: 2,  
    title: "Batman",  
    image: "/img/batman.jpg",  
    price: "$14.99",  
    isAvailable: false,  
  },  
];  
  
function App() {  
  return (  
    <>  
      <Header />  
      <Main comics={comics} />  
    </>  
  );  
}
```

```
    </>
  );
}

function Main({ comics }) {
  return (
    <main>
      <Section title="Current Series">
        <ComicsList comics={comics} />
      </Section>
    </main>
  );
}

function Section({ title, children }) {
  return (
    <section>
      <div className="container">
        <h2>{title}</h2>
        {children}
      </div>
    </section>
  );
}

function ComicsList({ comics }) {
  return (
    <div className="comics-grid">
      {comics.map((comic) => (
        <ComicCard
          key={comic.id}
          title={comic.title}
          image={comic.image}
          price={comic.price}
          isAvailable={comic.isAvailable}
        />
      ))}
    </div>
  );
}

function ComicCard({ title, image, price, isAvailable }) {
  return (
    <article className="comic-card">
      <img src={image} alt={title} />
      <h3>{title}</h3>
      <p>{price}</p>
      {isAvailable ? (
        <span className="available">Disponibile</span>
      ) : (
        <span className="not-available">Non disponibile</span>
      )}
    </article>
  );
}
```

```
);  
}
```

Errori Comuni

Dimenticare le graffe per valori JavaScript

Errato:

```
<ProductCard price="19.99" isAvailable="true" />
```

`price` e `isAvailable` qui sono stringhe.

Corretto:

```
<ProductCard price={19.99} isAvailable={true} />
```

Cercare di leggere props senza riceverle

Errato:

```
function Card() {  
  return <h3>{title}</h3>;  
}
```

Corretto:

```
function Card({ title }) {  
  return <h3>{title}</h3>;  
}
```

Usare `class` invece di `className`

Errato:

```
<div class="card"></div>
```

Corretto:

```
<div className="card"></div>
```

Passare troppe props non necessarie

Meno chiaro:

```
<ComicCard comic={comic} />
```

```
function ComicCard({ comic }) {  
  return (  
    <article>  
      <h3>{comic.title}</h3>  
      <img src={comic.image} alt={comic.title} />  
    </article>  
  );  
}
```

Piu esplicito per esercizi semplici:

```
<ComicCard  
  title={comic.title}  
  image={comic.image}  
/>
```

```
function ComicCard({ title, image }) {  
  return (  
    <article>  
      <h3>{title}</h3>  
      <img src={image} alt={title} />  
    </article>  
  );  
}
```

Nota: passare l'intero oggetto non e sempre sbagliato, ma nelle prime lezioni puo essere meno didattico se il componente usa solo due campi.

Confondere props e state

Per ora:

```
props = dati ricevuti dal padre  
state = dati interni che cambiano nel tempo
```

`useState` arriva nella prossima parte del corso. Per la lezione sulle props conviene concentrarsi su componenti controllati dall'esterno.

Domande da Farsi Durante un Esercizio

Quando creo un componente, posso chiedermi:

- Questo pezzo di UI si ripete?
- Quali parti cambiano tra una versione e l'altra?
- Quelle parti possono diventare props?
- Il componente deve decidere qualcosa da solo o deve ricevere tutto dal padre?
- Sto passando dati semplici e leggibili?
- Il nome delle props spiega bene cosa rappresentano?

Esempio mentale:

```
Card fumetto  
- struttura sempre uguale: immagine, titolo, prezzo  
- dati variabili: image, title, price  
- quindi image, title e price diventano props
```

Mini Glossario

Prop

Dato passato da un componente padre a un componente figlio.

Props object

L'oggetto che contiene tutte le props ricevute da un componente.

Destructuring

Sintassi JavaScript per estrarre direttamente valori da un oggetto.

Children

Prop speciale che contiene il JSX inserito tra apertura e chiusura di un componente.

Conditional rendering

Tecnica per mostrare JSX diverso in base a una condizione.

Dumb component

Componente che riceve dati e istruzioni dall'esterno e si limita a renderizzare la UI.

Composizione

Costruire interfacce combinando componenti piccoli dentro componenti più grandi.

Riassunto Finale

Le props sono il meccanismo principale per rendere i componenti React riutilizzabili.

Un buon componente con props:

- ha un nome chiaro;
- riceve solo i dati che gli servono;
- non modifica le props;
- renderizza JSX leggibile;
- puo essere usato piu volte con dati diversi;
- separa la struttura dai contenuti variabili.

Formula da ricordare:

`stessa UI + dati diversi = componente con props`